Comprehensive Backend Documentation

This manual is produced by scanning the current repository structure, route definitions, controllers, utilities, deployment files, and bundled datasets. It documents the backend as implemented in code.

- Repository root: C:\Users\rajus\fitlife\backend
- Python source files analyzed: 60
- Layer counts: 13 controllers, 13 route modules, 12 models, 11 utility modules, 5 schemas.
- Bundled dataset: final_dataset.csv with 5000 data rows
- Bundled dataset: final_dataset_BFP.csv with 5000 data rows

1. Project Purpose, Goals, and Scope

FitLife is a health, nutrition, and fitness backend that supports user authentication, health-profile capture, adaptive recommendations, AI-assisted food scanning, dashboard aggregation, progress tracking, reminder management, PDF export, and adaptive workout planning.

The central product idea is personalization. Age, gender, height, weight, activity level, food habits, BMI, estimated body-fat percentage, and fitness goal all influence calorie targets, recommendation content, AI planner responses, and workout-plan generation.

- Primary domains: authentication, health profiling, activity logging, food intelligence, workout planning, reminders, progress, recommendations, exports, and provider directories.
- Advanced capstone features in the repository include dataset-backed goal-period estimation, imported exercise media, workout sessions, and persisted scan images.

2. Architecture and Layering

The backend follows a standard Flask application-factory architecture with a thin route layer, controller-driven business logic, SQLAlchemy models, utility modules for algorithms and AI integration, and Marshmallow schemas for request validation.

```
Client / Frontend
    |
    v
Flask Blueprint Route -> Validation Middleware -> JWT Middleware -> Controller
    |
    +--> SQLAlchemy Models / PostgreSQL
    +--> Utility Modules (calculators, AI, planners, PDF generation)
    +--> JSON / PDF / Image Response
```

- Entry point: app.py creates the app, loads config, initializes extensions, registers blueprints, and defines global error handlers plus /health.
- WSGI entry: wsgi.py exposes create_app('production') for Gunicorn and Render.
- Configuration: config.py resolves DB URI, secrets, CORS origins, and API integration keys.
- Extensions: extensions.py centralizes SQLAlchemy, JWT, bcrypt, CORS, migrations, rate limiting, and APScheduler.

3. Setup, Configuration, and Deployment

Deployment and setup details were verified from README.md, config.py, render.yaml, and wsgi.py.

- Install dependencies: pip install -r requirements.txt
- Local run path documented in README: python app.py
- Production start command from render.yaml: gunicorn wsgi:app
- Health endpoint: /health

Environment variables discovered directly from code:

- GEMINI_API_KEY: Referenced in code; inspect implementation for exact behavior.
- GEMINI_VISION_MODEL: Referenced in code; inspect implementation for exact behavior.
- GROQ_API_KEY: Groq API key used by chatbot and scanner fallback.
- GROQ_TEXT_MODEL: Optional override for the Groq chat model.
- GROQ_VISION_MODEL: Referenced in code; inspect implementation for exact behavior.

Render service definition summary:

```
services: - type: web name: fitlife-backend runtime: python plan: free buildCommand: pip install -r requirements.txt startCommand: gunicorn wsgi:app healthCheckPath: /health autoDeploy: true envVars: - key: FLASK_ENV value: production - key: SECRET_KEY generateValue: true - key: JWT_SECRET_KEY generateValue: true - key: DATABASE_URL sync: false - key: GROQ_API_KEY sync: false - key: FRONTEND_ORIGIN sync: false
```

4. Tech Stack and Dependency Explanation

Core Framework

HTTP API framework, request/response handling, and app wiring.

- Flask==3.0.2
- Werkzeug==3.0.1

Database

ORM, migrations, and PostgreSQL connectivity.

- Flask-SQLAlchemy==3.1.1
- Flask-Migrate==4.0.5
- psycopg[binary]==3.2.13
- cryptography==42.0.5

Auth

Password hashing and JWT token support.

- Flask-JWT-Extended==4.6.0
- Flask-Bcrypt==1.0.1

Validation

Schema-driven request validation.

- marshmallow==3.21.1
- Flask-Marshmallow==1.2.0

CORS

Cross-origin browser access for the frontend.

- Flask-CORS==4.0.0

Rate Limiting

Per-IP throttling for abuse control.

- Flask-Limiter==3.5.0

AI (Groq - FREE)

Model inference used by the AI planner and scanner fallback path.

- groq==0.4.2

PDF Generation

Creates exportable health reports and documentation PDFs.

- fpdf2==2.7.9

Scheduler (reminders)

Background scheduling support.

- APScheduler==3.10.4

Environment

Loads .env values into runtime config.

- python-dotenv==1.0.1

HTTP

Outgoing calls to remote AI and dataset endpoints.

- requests==2.31.0

Development only

Testing and development support.

- pytest==8.1.1
- pytest-flask==1.3.0

Deployment

Production WSGI server.

- gunicorn==22.0.0

5. Data Model and Persistence

The backend persists application state in PostgreSQL through SQLAlchemy models.

Activ

One log entry: meal, workout, water, or sleep.

- Fields: id, user_id, log_date, log_type, description, calories_in, calories_out, water_ml, sleep_hours, duration_min, image_blob, image_mime_type, image_filename, created_at
- Methods: to_dict

Body

Reference records used by the goal estimator to approximate timelines.

- Fields: id, source, gender, age, height_m, weight_kg, bmi, bmi_case, body_fat_percentage, body_fat_case, recommendation_plan, created_at
- Methods: to_dict

Docto

Doctor listings for the doctor-help feature.

- Fields: id, name, specialization, hospital, contact_email, contact_phone, available_slots, rating, created_at
- Methods: to_dict

Exerc

Canonical exercise library imported from free-exercise-db.

- Fields: id, external_id, name, slug, source, level, category, force, mechanic, equipment, primary_muscles, secondary_muscles, instructions, image_urls, image_url, demo_media_url, tags, created_at, updated_at
- Methods: to_dict

Food

Food nutrition database - searchable by name or barcode.

- Fields: id, name, calories_per_100g, protein_g, carbs_g, fat_g, fiber_g, barcode, source
- Methods: to_dict

Healt

Stores health metrics and fitness goals for a user.
- Fields: id, user_id, age, gender, height_cm, weight_kg, activity_level, sleep_hours, food_habits, fitness_goal, bmi, body_fat_percentage, body_fat_category, bmr, daily_calories, created_at, updated_at
- Methods: to_dict

Reco

AI-generated diet and workout recommendations for a user.
- Fields: id, user_id, diet_plan, workout_plan, daily_calories, weekly_tips, bmi_category, generated_at
- Methods: to_dict

Remi

User reminders for workouts, meals, water, sleep, or custom alerts.
- Fields: id, user_id, reminder_type, message, remind_at, repeat_daily, is_active, created_at
- Methods: to_dict

Train

Fitness trainer listings for the trainer-connect feature.
- Fields: id, name, specialization, location, contact_email, contact_phone, rating, available, created_at
- Methods: to_dict

User

Represents an authenticated user account.
- Fields: id, full_name, email, password_hash, role, created_at, updated_at, profile, activity_logs, recommendations, workout_plans, reminders
- Methods: to_dict

Work

User's custom workout plan for a specific day of the week.
- Fields: id, user_id, plan_name, day_of_week, exercises, created_at, updated_at
- Methods: to_dict

Work

Tracks timer-driven workout execution for the advanced workout flow.
- Fields: id, user_id, status, day, goal, session_title, plan_snapshot, current_exercise_index, completed_exercises, total_duration_seconds, total_calories_burned, started_at, completed_at, updated_at
- Methods: to_dict

Migration files currently tracked:
- 0a1f2b3c4d5e_auth_session_placeholder.py
- b7f3029f4a21_add_advanced_workout_library.py
- e1f4dbb95c12_add_activity_log_scan_images.py
- f5b58c0adf38_initial_migration.py

6. API Surface

The endpoint list below is generated from route decorators inside the repository.

Doma

[POST] /api/activity -> activity_log
[GET] /api/activity -> activity_get
[GET] /api/activity/<int:log_id>/image -> activity_image_get

Domain: /ai

[POST] /api/ai/diet-chat -> chat

Domain: /dashboard

[GET] /api/dashboard -> dashboard

Domain: /doctors

[GET] /api/doctors -> doctors

Domain: /export

[GET] /api/export/pdf -> pdf_export

Domain: /food

[POST] /api/food/analyze-photo -> food_analyze_photo
[POST] /api/food/scan -> food_scan
[GET] /api/food/search -> food_search

Domain: /login

[POST] /api/login -> login

Domain: /logout

[POST] /api/logout -> logout

Domain: /profile

[GET] /api/profile -> profile_get
[POST] /api/profile -> profile_save

Domain: /progress

[GET] /api/progress -> progress

Domain: /recommendations

[GET] /api/recommendations -> recommendations

Domain: /register

[POST] /api/register -> register

Domain: /reminders

[GET] /api/reminders -> reminders_list
[POST] /api/reminders -> reminders_add
[DELETE] /api/reminders/<int:reminder_id> -> reminders_delete

Domain: /trainers

[GET] /api/trainers -> trainers

Domain: /workout

[GET] /api/workout/plan -> workout_get
[POST] /api/workout/plan -> workout_save
[POST] /api/workout/session/<int:session_id>/complete -> workout_session_complete
[POST] /api/workout/session/<int:session_id>/exercise-complete -> workout_session_complete_exercise
[POST] /api/workout/session/<int:session_id>/reset -> workout_session_reset
[GET] /api/workout/session/active -> workout_session_active
[POST] /api/workout/session/start -> workout_session_start
[POST] /api/workout/timer -> workout_timer

7. Major Workflows and Data Flow

Authentication and bootstrap

- POST /api/register validates credentials, normalizes email, hashes password, and creates a User record.

- POST /api/login validates credentials, checks the bcrypt hash, and returns a JWT access token.

- Protected feature routes use jwt_required_custom, which validates the token and injects current_user into controllers.

Health profile and recommendation refresh

- POST /api/profile normalizes onboarding labels, calculates BMI/BMR/TDEE/daily calories, estimates body-fat percentage, and stores HealthProfile.
- The same save flow refreshes Recommendation content and clears active workout sessions so planning stays consistent after profile changes.

Workout planning and execution

- GET /api/workout/plan checks custom WorkoutPlan rows first; if absent, it generates a profile-driven plan from the exercise library.
- The planner combines goal-track templates, exercise scoring, goal-timeline estimation, and media-backed exercise records.
- Workout sessions can be started, progressed, completed, or reset, with ActivityLog entries written for completed or timed workouts.

Food scanning and meal logging

- POST /api/food/analyze-photo validates image size and MIME type, tries structured AI analysis, enriches with local food data, and falls back to keyword catalogs.
- When logging is requested, a meal ActivityLog is created and the uploaded image bytes are persisted alongside the record.
- GET /api/activity and GET /api/activity/<id>/image expose that history back to the frontend.

Dashboard and progress aggregation

- GET /api/dashboard summarizes current-day calories, water, streaks, weekly chart data, and motivational content.
- GET /api/progress builds chart-ready weekly or monthly trend data from ActivityLog plus profile state.
- GET /api/export/pdf converts profile and recent activity data into a PDF report.

AI planner workflow

- POST /api/ai/diet-chat detects a topic, loads profile and recent activity context, optionally pulls a workout plan, and builds a system prompt for Groq.
- If Groq is unavailable, the controller returns a rule-based fallback response that remains personalized from profile and activity context.

```
Profile save -> HealthProfile updated -> BMI/BFP recalculated -> Recommendation refreshed ->
Workout plan generated on demand -> Session execution -> ActivityLog updated -> Dashboard/Progress
refreshed
```

8. Key Algorithms, Technical Terms, and Concepts

- BMI: Body Mass Index; implemented in utils/bmi_calculator.py as weight divided by squared height in meters.
- BMR: Basal Metabolic Rate; estimated with the Mifflin-St Jeor equation.
- TDEE: Total Daily Energy Expenditure; derived from BMR multiplied by an activity-level coefficient.
- Daily calorie target: Adjusted from TDEE according to fitness goal, with a minimum safety floor.
- Estimated body fat: Calculated with a Deurenberg-style heuristic in utils/body_fat.py.
- Goal-period estimation: Implemented in utils/goal_achievement_model.py using BMI/BFP reference rows plus heuristics to estimate weeks to target.
- Exercise scoring: Implemented in utils/workout_planner.py; exercises are ranked by category fit, muscle overlap, difficulty fit, and media availability.
- Structured AI output: Scanner utilities force Gemini or Groq to return strict JSON payloads for downstream nutrition processing.

- Fallback catalog: Keyword-to-food mappings used when vision is uncertain to keep the scanner usable in real-world conditions.
- Persisted scan image: Meal activity logs can store uploaded image bytes directly in the database and expose them through an authenticated endpoint.

Algorithm highlights verified from repository code:

- utils/goal_achievement_model.py computes target BMI/BFP values, safe weekly change rates, candidate distance against reference rows, and a weighted estimated-week result.
- utils/workout_planner.py defines per-goal weekly tracks, maps user activity level to difficulty ceilings, scores exercise-library entries, prescribes sets/reps/duration/rest, and composes today/weekly plan payloads.
- controllers/food_controller.py combines vision models, DB matching, fallback catalogs, macro scaling, scan recovery guidance, and goal-aware warnings.
- controllers/ai_controller.py lazily loads workout context only for relevant topics to keep responses efficient.

9. File-by-File Breakdown

app.py

FitLife Backend - Entry Point Flask application factory with all extensions, blueprints, and error handlers.

- Functions: create_app

wsgi.py

Top-level backend file: wsgi.py

config.py

Top-level backend file: config.py

extensions.py

Top-level backend file: extensions.py

README.md

Getting-started guide, endpoint summary, and project notes. Some text is historical and no longer reflects newer backend upgrades.

render.yaml

Render deployment descriptor that defines build command, start command, plan, health check, and key environment variables.

requirements.txt

Pinned Python dependencies grouped by responsibility.

Directory: controllers

controllers\activity_controller.py

controllers module.

- Functions: log_activity, get_activity, get_activity_image

contr

AI planner controller with strong profile-aware fallback behavior.

- Functions: diet_chat, _build_chat_context, _build_system_prompt, _detect_topic, _meal_text, _format_today_workout, _fallback_response, _build_response

contr

controllers module.

- Functions: register_user, login_user

contr

controllers module.
- Functions: get_dashboard, _calculate_streak

contr

controllers module.
- Functions: list_doctors

contr

controllers module.
- Functions: export_pdf

contr

controllers module.
- Functions: search_food, _find_food, _normalize_lookup_text, _catalog_match, _coarse_food_identification, _scaled_macros, _build_food_feedback, _goal_display_label, _build_goal_warning, _fallback_analysis, _extract_serving_grams, _extract_number

contr

controllers module.
- Functions: get_profile, save_profile, _refresh_recommendations

contr

controllers module.
- Functions: get_progress

contr

controllers module.
- Functions: _legacy_workout_map, _upsert_recommendation, get_recommendation

contr

controllers module.
- Functions: list_reminders, add_reminder, delete_reminder

contr

controllers module.
- Functions: list_trainers

contr

controllers module.
- Functions: _serialize_custom_plans, _active_session_payload, get_plan, save_plan, log_timer, start_session, get_active_session, complete_exercise, complete_session, reset_session

Directory: routes

routes\activity_routes.py

routes module.
- Functions: activity_log, activity_get, activity_image_get
- Endpoints: /api/activity, /api/activity, /api/activity/<int:log_id>/image

route

routes module.
- Functions: chat
- Endpoints: /api/ai/diet-chat

route

routes module.
- Functions: register, login, logout
- Endpoints: /api/register, /api/login, /api/logout

route

routes module.

- Functions: dashboard
- Endpoints: /api/dashboard

route

routes module.
- Functions: doctors
- Endpoints: /api/doctors

route

routes module.
- Functions: pdf_export
- Endpoints: /api/export/pdf

route

routes module.
- Functions: food_search, food_scan, food_analyze_photo
- Endpoints: /api/food/search, /api/food/scan, /api/food/analyze-photo

route

routes module.
- Functions: profile_get, profile_save
- Endpoints: /api/profile, /api/profile

route

routes module.
- Functions: progress
- Endpoints: /api/progress

route

routes module.
- Functions: recommendations
- Endpoints: /api/recommendations

route

routes module.
- Functions: reminders_list, reminders_add, reminders_delete
- Endpoints: /api/reminders, /api/reminders, /api/reminders/<int:reminder_id>

route

routes module.
- Functions: trainers
- Endpoints: /api/trainers

route

routes module.
- Functions: workout_get, workout_save, workout_timer, workout_session_start, workout_session_active, workout_session_complete_exercise, workout_session_complete, workout_session_reset
- Endpoints: /api/workout/plan, /api/workout/plan, /api/workout/timer, /api/workout/session/start, /api/workout/session/active, /api/workout/session/<int:session_id>/exercise-complete, /api/workout/session/<int:session_id>/complete, /api/workout/session/<int:session_id>/reset

Directory: models

models\activity_log.py

models module.
- Classes: ActivityLog

mode

models module.
- Classes: BodyMetricReference

mode

models module.
- Classes: Doctor

mode

models module.
- Classes: ExerciseLibrary

mode

models module.
- Classes: FoodItem

mode

models module.
- Classes: HealthProfile

mode

models module.
- Classes: Recommendation

mode

models module.
- Classes: Reminder

mode

models module.
- Classes: Trainer

mode

models module.
- Classes: User

mode

models module.
- Classes: WorkoutPlan

mode

models module.
- Classes: WorkoutSession

Directory: utils

utils\ai_structured.py

Helpers for JSON-only Gemini/Groq responses and graceful fallback parsing.
- Functions: _strip_json_wrappers, _parse_json_output, _post_chat_completion, _post_gemini_generate_content, groq_json_vision, gemini_json_vision, gemini_nutrition_vision

utils\b

BMI, BMR, TDEE, and daily calorie calculations. Uses the Mifflin-St Jeor equation - most accurate for most people.
- Functions: calculate_bmi, get_bmi_category, calculate_bmr, calculate_tdee, calculate_daily_calories

utils\b

Body-fat estimation helpers used for profile-aware workout planning.
- Functions: estimate_body_fat_percentage, body_fat_category

utils\c

Pre-defined meal templates per goal and food preference. Keys: low_cal, high_protein, balanced Sub-keys: non-veg, veg,

vegan Each meal: {meal: name, kcal: calories}

utils\g

A lightweight model-backed estimator for fitness goal timelines.

- Functions: _activity_day_target, _goal_target_bmi, _target_body_fat, _safe_weekly_change, _candidate_distance, _reference_matches, estimate_goal_timeline

utils\p

PDF health report generator using fpdf2. Generates a styled health report with body metrics and weekly activity summary.

- Functions: generate_health_report

utils\q

Daily motivational quotes. Rotates based on day-of-year so each day gets a consistent quote.

- Functions: get_daily_quote

utils\r

Rule-based recommendation engine. Selects diet template + workout template + weekly tips based on BMI, fitness goal, and food preference.

- Functions: generate_recommendation

utils\v

Custom validation helpers for data sanitization and parsing.

- Functions: normalize_email, parse_time_string, parse_date_string, sanitize_string

utils\w

Profile-driven workout planning built on the exercise library.

- Functions: _today_key, _normalize, _exercise_score, _select_exercises, _prescribe_exercise, _build_rest_day, generate_profile_workout_plan

utils\w

Goal-based workout templates with posture guidance and calorie estimates.

- Functions: goal_to_workout_key, _estimate_duration_min, _estimate_calories_burn, enrich_exercise, build_workout_day, build_goal_based_workout_plan

Directory: schemas

schemas\activity_schema.py

schemas module.

- Classes: ActivityLogSchema

schem

schemas module.

- Classes: RegisterSchema, LoginSchema

schem

schemas module.

- Classes: HealthProfileSchema

schem

schemas module.

- Classes: ReminderSchema

schem

schemas module.

- Classes: ExerciseSchema, WorkoutPlanSchema

Directory: middleware

middleware\auth_middleware.py

middleware module.

- Functions: jwt_required_custom

middl

middleware module.

- Functions: validate_body

Directory: scripts

scripts\generate_backend_manual_pdf.py

Generate a comprehensive backend technical manual PDF from repository analysis.

- Classes: ManualPDF

- Functions: read_text, summarize_text, pdf_safe, is_sqlalchemy_field, parse_class, parse_route_decorators, scan_python_file, parse_requirements, dataset_rows, scan_repository, add_cover, add_project_overview

script

Generate a project-overview PDF for the FitLife backend.

- Classes: ProjectPDF

- Functions: build_pdf

script

Import BMI/BFP reference datasets stored inside the repo.

- Functions: _clean_gender, _to_float, _to_int, import_csv, main

script

Import exercises from free-exercise-db into the exercise library table.

- Functions: _slugify, _image_url, load_dataset, main

10. Assumptions, Limitations, and Edge Cases

Assumptions and limitations identified directly from code and repository contents:

- README.md still contains older wording such as MySQL references and an older endpoint count, while the live code targets PostgreSQL/Neon and a richer feature set.
- Body-fat percentage is estimated heuristically; it is not a clinically measured value.
- Goal-period estimation is model-backed but lightweight. It uses reference datasets plus heuristics instead of a separately trained model artifact.
- Food scanning quality depends on upstream AI provider availability, image clarity, package readability, and fallback catalog coverage.
- Some AI integrations are environment-dependent. Runtime availability can vary by provider key, model support, quota, or geographic restrictions.
- The free exercise import script depends on either a local clone path or a remote GitHub dataset URL being reachable at import time.
- Scheduler configuration exists, but the repository does not define a persistent APScheduler job store within this codebase.
- The repository does not currently contain a broad automated test suite; correctness is driven mainly by runtime behavior and validation guards.

Important edge cases handled in code:

- Missing health profile: profile, workout, and AI planner routes prompt profile creation.
- Invalid JWT: jwt_required_custom returns a standardized 401 response.
- Unclear food photo: scanner falls back from structured AI to coarse identification to DB/catalog matching and finally scan-recovery hints.
- No active workout session: session lookup returns 404 rather than silent success.
- Custom workout plan present: workout controller prefers custom plans over generated plans.
- Scan image retrieval without request context: ActivityLog.to_dict falls back to relative or configurable API URL

generation.